

RELACIÓN DE PROBLEMAS DE LA UD3 – CS2 DAW

1. A partir de la información siguiente, implementa la clase en PHP correspondiente, además de lo necesario para probar su ejecución y prueba:

Persona:

- Atributos:
 - - nombre: cadena de caracteres.
 - - edad: numérica.
 - + familia: cadena de caracteres
- Comportamiento:
 - Los necesarios para encapsular sus atributos.
 - El método saludar, que mostrará: "Hola, soy {nombre}, tengo edad} años y soy de la familia {familia}."

```
<?php
class Persona {
    private $nombre;
    private $edad;
    public $familia;

    public function __construct($nombre, $edad, $familia) {
        $this->nombre = $nombre;
        $this->edad = $edad;
        $this->familia = $familia;
    }

    public function getNombre() {
        return $this->nombre;
    }

    public function setEdad($edad) {
        $this->edad = $edad;
    }

    public function saludar() {
        echo "Hola, soy {$this->nombre}, tengo {$this->edad} años y".
            "soy de la familia {$this->familia}.";
    }
}

// lo probamos
$p = new Persona("Ana", 25, "Espabilados");
$p->saludar();
echo " ".<br>";

$p->setEdad(26);
$p->saludar();
echo " ".<br>";

echo "La familia es: {$p->familia} ".<br>";
echo " ".<br>";
?>
```

2. Modificar la clase del ejercicio 1 para implementar la sobrecarga de dos constructores, uno con la información al instanciar el objeto y otra sin valores. Implementa la clase en PHP correspondiente, además de lo necesario para probar su ejecución y prueba:

```
<?php
class Persona {
    private $nombre;
    private $edad;
    public $familia;

    public function __construct($nombre=null, $edad=null, $familia=null) {
        $this->nombre = $nombre;
        $this->edad = $edad;
        $this->familia = $familia;
    }

    public function setNombre($nombre) {
        $this->nombre = $nombre;
    }

    public function getNombre() {
        return $this->nombre;
    }

    public function setEdad($edad) {
        $this->edad = $edad;
    }

    public function getEdad() {
        return $this->edad;
    }

    public function saludar() {
        echo "Hola, soy {$this->nombre}, tengo {$this->edad} años y".
            " soy de la familia {$this->familia}.";
    }
}

// lo probamos con valores en el constructor
$p1 = new Persona("Ana", 25, "Espabilados");
$p1->saludar();
echo " ".<br>";

$p1->setEdad(26);
$p1->saludar();
echo " ".<br>";

echo "La familia es: {$p1 -> familia} ".<br>";
echo " ".<br>";

// lo probamos sin valores en el constructor
$p2 = new Persona();
$p2->saludar();
echo " ".<br>";

$p2->setNombre("Antonio");
$p2->setEdad(30);
$p2->familia = "Normales";
$p2->saludar();
```

```

echo " ".<br>";

echo "La familia es: {$p2 -> familia} ".<br>";
echo "Fin del script ".<br>";
?>

```

3. Modificar la clase del ejercicio 2 para añadir un destructor, en el que indique un mensaje para confirmar que el objeto se ha destruido.

```

<?php
class Persona {
    private $nombre;
    private $edad;
    public $familia;

    public function __construct($nombre=null, $edad=null, $familia=null) {
        $this->nombre = $nombre;
        $this->edad = $edad;
        $this->familia = $familia;
    }

    public function setNombre($nombre) {
        $this->nombre = $nombre;
    }

    public function getNombre() {
        return $this->nombre;
    }

    public function setEdad($edad) {
        $this->edad = $edad;
    }

    public function getEdad() {
        return $this->edad;
    }

    public function saludar() {
        echo "Hola, soy {$this->nombre}, tengo {$this->edad} años y".
            " soy de la familia {$this->familia}.";
    }

    public function __destruct() {
        echo "El objeto correspondiente a la persona {$this->nombre} ha sido destruido".<br>";
    }
}

// lo probamos con valores en el constructor
$p1 = new Persona("Ana", 25, "Espabilados");
$p1->saludar();
echo " ".<br>";

$p1->setEdad(26);
$p1->saludar();
echo " ".<br>";

echo "La familia es: {$p1 -> familia} ".<br>";
echo " ".<br>";

```

```

// lo probamos sin valores en el constructor
$p2 = new Persona();
$p2->saludar();
echo " ".<br>";

$p2->setNombre("Antonio");
$p2->setEdad(30);
$p2->familia = "Normales";
$p2->saludar();
echo " ".<br>";

echo "La familia es: {$p2 -> familia}."<br>";
echo "Fin del script "."<br>";
?>

```

4. Modificar la clase del ejercicio 3 para implementar la sobrecarga del constructor, de forma que los parámetros sean variables, los no indicados se asumirá un valor por defecto. Implementa la clase en PHP correspondiente, además de lo necesario para probar su ejecución y prueba:

```

<?php
class Persona {
    private $nombre;
    private $edad;
    public $familia;

    public function __construct($nombre=null, $edad=null, $familia=null) {
        $numArgs = func_num_args();
        $args = func_get_args();

        if ($numArgs == 1) {
            $this->nombre = $args[0];
            $this->edad = 0;
            $this->familia = "Sin familia";
        } elseif ($numArgs == 2) {
            $this->nombre = $args[0];
            $this->edad = $args[1];
            $this->familia = "Sin familia";
        } elseif ($numArgs == 3) {
            $this->nombre = $args[0];
            $this->edad = $args[1];
            $this->familia = $args[2];
        } else {
            $this->nombre = "Sin nombre";
            $this->edad = 0;
            $this->familia = "Sin familia";
        }
    }

    public function setNombre($nombre) {
        $this->nombre = $nombre;
    }

    public function getNombre() {
        return $this->nombre;
    }

    public function setEdad($edad) {

```

```

        $this->edad = $edad;
    }

    public function getEdad() {
        return $this->edad;
    }

    public function saludar() {
        echo "Hola, soy {$this->nombre}, tengo {$this->edad} años y".
            " soy de la familia {$this->familia}.";
    }

    public function __destruct() {
        echo "El objeto correspondiente a la persona {$this->nombre} ha sido destruido". "<br>";
    }
}

// lo probamos con valores en el constructor, persona1
$p1 = new Persona("Ana", 25, "Espabilados");
$p1->saludar();
echo " ". "<br>";

$p1->setEdad(26);
$p1->saludar();
echo " ". "<br>";

echo "La familia es: {$p1 -> familia} ". "<br>";
echo " ". "<br>";

// person2 sin valores en el constructor
$p2 = new Persona();
$p2->saludar();
echo " ". "<br>";

$p2->setNombre("Antonio");
$p2->setEdad(30);
$p2->familia = "Normales";
$p2->saludar();
echo " ". "<br>";

echo "La familia es: {$p2 -> familia} ". "<br>";

// persona 3
$p3 = new Persona("María");
$p3->saludar();
echo " ". "<br>";

$p3->setEdad(30);
$p3->familia = "Parcial";
$p3->saludar();
echo " ". "<br>";

echo "La familia es: {$p3 -> familia} ". "<br>";

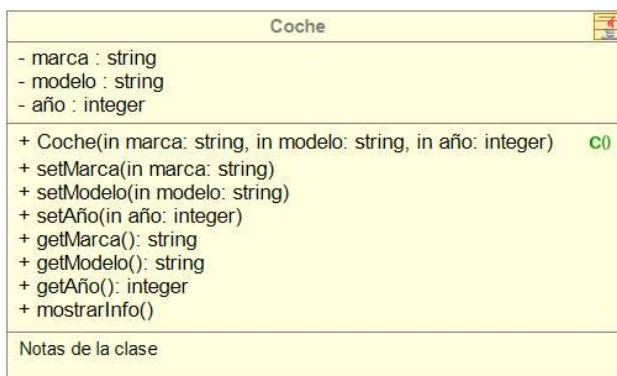
echo "Fin del script ". "<br>";
?>

```

5. Se desea modelar una clase Coche para recoger estas características de la clase y este comportamiento:

- Atributos:
 - Marca, la consideraremos de tipo cadena.
 - Modelo, la consideraremos de tipo cadena.
 - Año, la consideraremos de tipo entero.
- Métodos:
 - Constructor de la clase que reciba los valores iniciales del mismo en su creación.
 - Establecer nuevos valores: setCoche con los parámetros necesarios.
 - Obtener valores del coche: getCoche() que devolverá un objeto Coche.
 - Visualizar valores del coche: mostrarInfo() que imprima en pantalla la información del coche.

La cual está modelada en un diagrama de clases de esta forma:



Implementar en PHP la clase con lo necesario para poder probar su ejecución y prueba.

```
<?php
class Coche {
    // Atributos privados
    private $marca;
    private $modelo;
    private $año;

    // Constructor: inicializa la clase con los valores recibidos
    public function __construct($marca, $modelo, $año) {
        $this->marca = $marca;
        $this->modelo = $modelo;
        $this->año = $año;
    }

    // Método para establecer nuevos valores al coche
    public function setCoche($marca, $modelo, $año) {
        $this->marca = $marca;
        $this->modelo = $modelo;
        $this->año = $año;
    }

    // Método que devuelve un nuevo objeto Coche con los valores actuales
    public function getCoche() {
        return new Coche($this->marca, $this->modelo, $this->año);
    }

    // Método que muestra la información del coche
    public function mostrarInfo() {
```

```

        echo "Marca: " . $this->marca . "<br>";
        echo "Modelo: " . $this->modelo . "<br>";
        echo "Año: " . $this->año . "<br>";
    }
}

// Ejemplo de uso
echo "Coche 1". "<br>";
$coche1 = new Coche("Toyota", "Corolla", 2020);
$coche1->mostrarInfo();

echo "<hr>";

// Cambiamos los valores con setCoche
echo "Coche 1 modificado". "<br>";
$coche1->setCoche("Ford", "Focus", 2022);
$coche1->mostrarInfo();

echo "<hr>";

// Obtenemos un nuevo objeto Coche a partir de getCoche
echo "Coche 2". "<br>";
$coche2 = $coche1->getCoche();
$coche2->mostrarInfo();
?>

```

6. Se desea modelar una clase CuentaBancaria para recoger la cantidad de cuentas que se tienen de forma estática, de forma que cada una disponga de su información particular, con estas características de la clase y este comportamiento:

- Atributos:
 - saldo: numérico.
 - numcuentas: numérico. Ha de ser estática para almacenar la cantidad de cuentas que se han instanciado.
- Métodos:
 - Constructor con el saldo inicial e incrementando la cantidad de cuentas.
 - Depositar saldo: incrementa el saldo de la cuenta instanciada.
 - Retirar saldo: decrementar el saldo de la cuenta instanciada.
 - Obtener saldo: obtener el saldo de la cuenta instanciada.
 - Obtener cantidad de cuentas: obtener las cuentas que hay del atributo estático.

Implementar en PHP la clase con lo necesario para poder probar su ejecución y prueba.

```

<?php
// clase para recoger la información horizontal a todas las cuentas
// se declara como estática

class CuentaBancaria { // cada objeto tendrá su saldo, pero la cantidad es horizontal
    private $saldo;
    private static $numCuentas = 0;

    public function __construct($saldoInicial) {
        $this->saldo = $saldoInicial;
        self::$numCuentas++;
    }

    public function depositar($cantidad) {
        $this->saldo += $cantidad;
    }
}

```

```

public function retirar($cantidad) {
    if (($this->saldo - $cantidad) >= 0){
        $this->saldo -= $cantidad;
    } else {
        $this->saldo = 0;
    }
}

public function getSaldo() {
    return $this->saldo;
}

public static function getNumCuentas() {
    return self::$numCuentas;
}
}

echo "Número de cuentas: " . CuentaBancaria::getNumCuentas(). "<br>";

$c1 = new CuentaBancaria(100);
echo "Saldo de la cuenta 1: " . $c1->getSaldo(). "<br>";
echo "Número de cuentas: " . CuentaBancaria::getNumCuentas(). "<br>";

$c2 = new CuentaBancaria(200);
echo "Saldo de la cuenta 2: " . $c2->getSaldo(). "<br>";
echo "Número de cuentas: " . CuentaBancaria::getNumCuentas(). "<br>";

$c3 = new CuentaBancaria(300);
echo "Saldo de la cuenta 3: " . $c3->getSaldo(). "<br>";
echo "Número de cuentas: " . CuentaBancaria::getNumCuentas(). "<br>";

?>

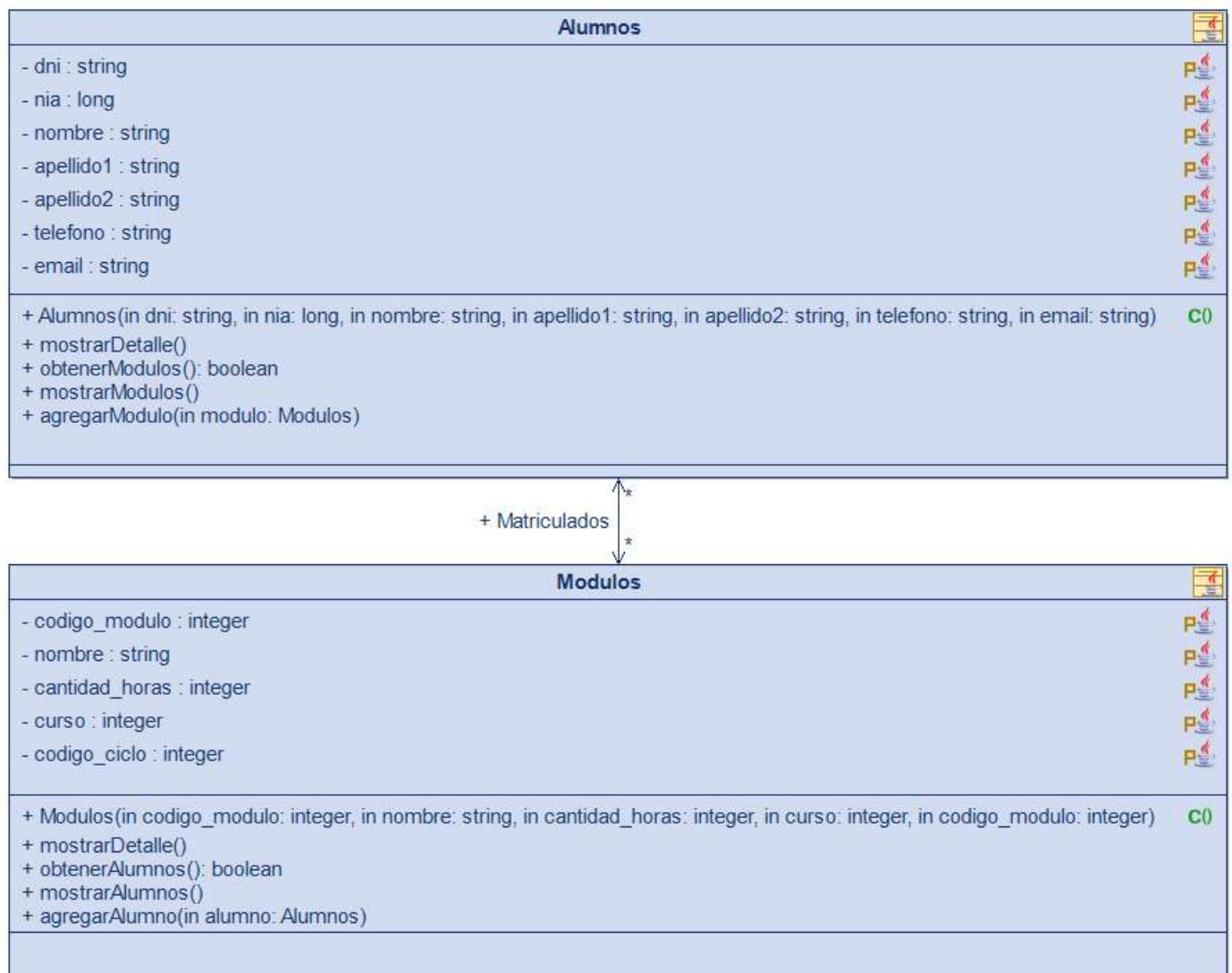
```

7. Crear una relación de asociación simple bidireccional. Para lo cual, crea dos clases:

- Alumno: un alumno puede estar matriculado en varios cursos.
 - Con estos atributos:
 - dni, cadena.
 - nia, entero largo.
 - nombre, apellido1 y apellido2 del tipo cadena.
 - telefono, cadena.
 - email, cadena.
 - Con estos métodos:
 - Constructor de la clase que reciba los valores iniciales del mismo en su creación, sobrecargado sin valores.
 - Establecer nuevos valores con los métodos set.
 - Obtener valores con los métodos get.
 - mostrarDetalle, mostrará la información del módulo.
 - agregarModulo, incrementará un módulo matriculado al alumno. A su vez, el módulo matriculado también conocerá al alumno (es bidireccional).
 - obtenerModulos, devuelve los módulos en los que está matriculado el alumno.
 - mostrarModulos, mostrará la información de los módulos matriculados por el alumno.
- Modulos: un módulo puede tener varios alumnos matriculados.
 - Con estos atributos:
 - codigo_modulo, entero.
 - nombre, cadena.
 - cantidad_horas, entero.
 - curso, entero.

- codigo_ciclo, entero.
- Con estos métodos:
 - Constructor de la clase que reciba los valores iniciales del mismo en su creación, sobrecargado sin valores.
 - Establecer nuevos valores con los métodos set.
 - Obtener valores con los métodos get.
 - mostrarDetalle, mostrará la información del módulo.
 - agregarAlumno, incrementará un alumno matriculado en el módulo. A su vez, el alumno matriculado también conocerá al módulo (es bidireccional).
 - obtenerAlumnos, obtendrá de la BD los alumnos matriculados en el módulo (simulándolo).
 - mostrarAlumnos, mostrará la información de los alumnos matriculados en el módulo.

Con esta información, se ha diseñado y modelado las clases de la siguiente forma (no se han representado algunos métodos indicados anteriormente):



Implementar en PHP las clases con lo necesario para poder probar su ejecución y prueba. Utilizar tipado fuerte (tipos en las variables, parámetros y métodos).

```

<?php
// Definimos la clase Alumno
class Alumno {
    // Atributos
    private string $dni;
  
```

```

private int $nia;
private string $nombre;
private string $apellido1;
private string $apellido2;
private string $telefono;
private string $email;
private array $modulos; // Asociación: para la lista de módulos

// Constructor, con valores por defecto
public function __construct(string $dni = "",
    int $nia = 0,
    string $nombre = "",
    string $apellido1 = "",
    string $apellido2 = "",
    string $telefono = "",
    string $email = "" ) {
    $this->dni = $dni;
    $this->nia = $nia;
    $this->nombre = $nombre;
    $this->apellido1 = $apellido1;
    $this->apellido2 = $apellido2;
    $this->telefono = $telefono;
    $this->email = $email;
    $this->modulos = []; // inicializamos la lista vacia, en este momento no hay módulos
}

// Setters
public function setDni(string $dni) { $this->dni = $dni; }
public function setNia(int $nia) { $this->nia = $nia; }
public function setNombre(string $nombre) { $this->nombre = $nombre; }
public function setApellido1(string $apellido1) { $this->apellido1 = $apellido1; }
public function setApellido2(string $apellido2) { $this->apellido2 = $apellido2; }
public function setTelefono(string $telefono) { $this->telefono = $telefono; }
public function setEmail(string $email) { $this->email = $email; }

// Getters
public function getDni(): string { return $this->dni; }
public function getNia(): int { return $this->nia; }
public function getNombre(): string { return $this->nombre; }
public function getApellido1(): string { return $this->apellido1; }
public function getApellido2(): string { return $this->apellido2; }
public function getTelefono(): string { return $this->telefono; }
public function getEmail(): string { return $this->email; }

// Métodos de relación, simulamos el no tener acceso a una BBDD
public function agregarModulo(Modulo $modulo) { // aquí es donde se indica el Modulo
    // Evitamos duplicados
    if (!in_array($modulo, $this->modulos, true)) {
        $this->modulos[] = $modulo;

        // Relación bidireccional: el módulo también conoce al alumno,
        // por lo que hay que agregarlo
        $modulo->agregarAlumno($this);
    }
}

// Mostrar detalle
public function mostrarDetalle() {

```

```

        echo "Alumno: {$this->nombre} {$this->apellido1} {$this->apellido2} (NIA: {$this->nia}, DNI:
{$this->dni})<br>";
        echo "Teléfono: {$this->telefono}, Email: {$this->email}<br>";
    }

    // Obtener módulos (simulado): cuando tengamos acceso a una BBDD la obtendremos de ahí.
    public function obtenerModulos(): array {
        return $this->modulos;
    }

    // Mostrar módulos
    public function mostrarModulos() {
        echo "Módulos matriculados por {$this->nombre}<br>";
        foreach ($this->modulos as $modulo) {
            $modulo->mostrarDetalle();
        }
        echo "<br>";
    }
}

class Modulo {
    // Atributos
    private int $codigo_modulo;
    private string $nombre;
    private int $cantidad_horas;
    private int $curso;
    private int $codigo_ciclo;
    private array $alumnos; // Asociación: lista de alumnos

    // Constructor con valores por defecto
    public function __construct(int $codigo_modulo = 0,
                                string $nombre = "",
                                int $cantidad_horas = 0,
                                int $curso = 0,
                                int $codigo_ciclo = 0 ) {
        $this->codigo_modulo = $codigo_modulo;
        $this->nombre = $nombre;
        $this->cantidad_horas = $cantidad_horas;
        $this->curso = $curso;
        $this->codigo_ciclo = $codigo_ciclo;
        $this->alumnos = []; // inicializamos la lista de alumnos que tienen el módulo
    }

    // Setters
    public function setCodigoModulo(int $codigo_modulo) { $this->codigo_modulo =
$codigo_modulo; }
    public function setNombre(string $nombre) { $this->nombre = $nombre; }
    public function setCantidadHoras(int $cantidad_horas) { $this->cantidad_horas =
$cantidad_horas; }
    public function setCurso(int $curso) { $this->curso = $curso; }
    public function setCodigoCiclo(int $codigo_ciclo) { $this->codigo_ciclo = $codigo_ciclo; }

    // Getters
    public function getCodigoModulo(): int { return $this->codigo_modulo; }
    public function getNombre(): string { return $this->nombre; }
    public function getCantidadHoras(): int { return $this->cantidad_horas; }
    public function getCurso(): int { return $this->curso; }
    public function getCodigoCiclo(): int { return $this->codigo_ciclo; }
}

```

```

// Métodos de relación
public function agregarAlumno(Alumno $alumno) {
    // Evitamos duplicados
    if (!in_array($alumno, $this->alumnos, true)) {
        $this->alumnos[] = $alumno;

        // Relación bidireccional: el alumno también conoce al módulo,
        // por lo que hay que agregarlo
        $alumno->agregarModulo($this);
    }
}

// Mostrar detalle
public function mostrarDetalle() {
    echo "Módulo: {$this->nombre} (Código: {$this->codigo_modulo})<br>";
    echo "Horas: {$this->cantidad_horas}, Curso: {$this->curso}, Ciclo: {$this->codigo_ciclo}<br>";
}

// Obtener alumnos
public function obtenerAlumnos(): array {
    return $this->alumnos;
}

// Mostrar alumnos
public function mostrarAlumnos() {
    echo "Alumnos matriculados en {$this->nombre}<br>";
    foreach ($this->alumnos as $alumno) {
        $alumno->mostrarDetalle();
    }
    echo "<br>";
}

// Ejemplo de uso: creamos los alumnos y módulos
$alumno1 = new Alumno("12345678A", 1001, "María", "López", "Pérez", "600111222",
"maria@example.com");
$alumno2 = new Alumno("98765432B", 1002, "Juan", "Martín", "García", "600333444",
"juan@example.com");

$mod1 = new Modulo(101, "Programación", 120, 1, 2001);
$mod2 = new Modulo(102, "Bases de Datos", 100, 1, 2001);

// Asociaciones bidireccionales
$alumno1->agregarModulo($mod1);
$alumno1->agregarModulo($mod2);
$alumno2->agregarModulo($mod1);

$mod2 ->agregarAlumno($alumno2);

// Visualización de los alumnos
echo "ALUMNO 1 ". "<br>";
$alumno1->mostrarDetalle();
echo " ". "<br>";
$alumno1->mostrarModulos();

echo "ALUMNO 2 ". "<br>";

```

```

$alumno2->mostrarDetalle();
echo " ". "<br>";
$alumno2->mostrarModulos();

// Visualización de los módulos
echo "MÓDULO 1: ". "<br>";
$mod1->mostrarDetalle();
echo " ". "<br>";
$mod1->mostrarAlumnos();

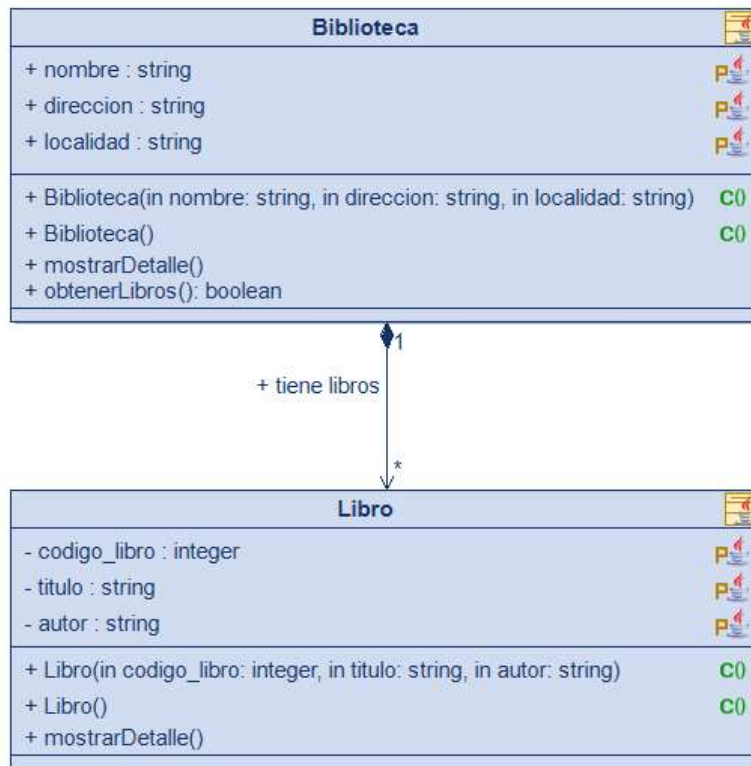
echo "MÓDULO 2: ". "<br>";
$mod2->mostrarDetalle();
echo " ". "<br>";
$mod2->mostrarAlumnos();
?>

```

8. Crear una relación de composición. Para lo cual, se crea una clase Biblioteca que contenga varias instancias de la clase Libro. La biblioteca puede existir sin libros, pero al revés no:

- Biblioteca: una biblioteca se compone de libros.
 - Con estos atributos:
 - nombre, cadena.
 - direccion, cadena.
 - localidad, cadena.
 - Con estos métodos:
 - Constructor de la clase que reciba los valores iniciales del mismo en su creación, sobrecargado sin valores.
 - Establecer nuevos valores con los métodos set.
 - Obtener valores con los métodos get.
 - mostrarDetalle, mostrará la información de la biblioteca.
 - obtenerLibros, obtendrá de la BD los libros que componen la biblioteca.
 - mostrarLibros, mostrará la información de los módulos matriculados por el alumno.
- Libros:
 - Con estos atributos:
 - codigo_libro, entero.
 - titulo, cadena.
 - autor, cadena.
 - Con estos métodos:
 - Constructor de la clase que reciba los valores iniciales del mismo en su creación, sobrecargado sin valores.
 - Establecer nuevos valores con los métodos set.
 - Obtener valores con los métodos get.
 - MostrarDetalle, mostrará la información del libro.

Con esta información, se ha diseñado y modelado las clases de la siguiente forma:

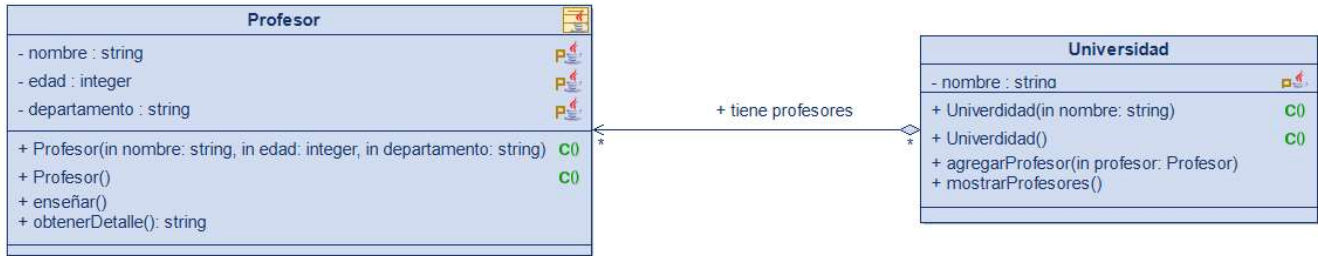


Implementar en PHP las clases con lo necesario para poder probar su ejecución y prueba.

9. Crear una relación de agregación. Para lo cual, modelar un sistema donde una Universidad tiene múltiples Profesores. La Universidad agrega a los Profesores, pero los Profesores pueden existir sin la Universidad.

- Universidad: tienen su lista de profesores.
 - Con estos atributos:
 - nombre, cadena.
 - Con estos métodos:
 - Constructor de la clase que reciba los valores iniciales del mismo en su creación, sobrecargado sin valores.
 - Establecer nuevos valores con los métodos set.
 - Obtener valores con los métodos get.
 - agregarProfesor, añade un profesor a su lista.
 - mostrarProfesores, mostrará los profesores de la universidad.
- Profesor:
 - Con estos atributos:
 - nombre, cadena.
 - edad, entero.
 - departamento, cadena.
 - Con estos métodos:
 - Constructor de la clase que reciba los valores iniciales del mismo en su creación, sobrecargado sin valores.
 - Establecer nuevos valores con los métodos set.
 - Obtener valores con los métodos get.
 - enseñar, mostrará lo que enseña el profesor.
 - obtenerDetalle, devuelve el detalle del profesor.

Con esta información, se ha diseñado y modelado las clases de la siguiente forma:

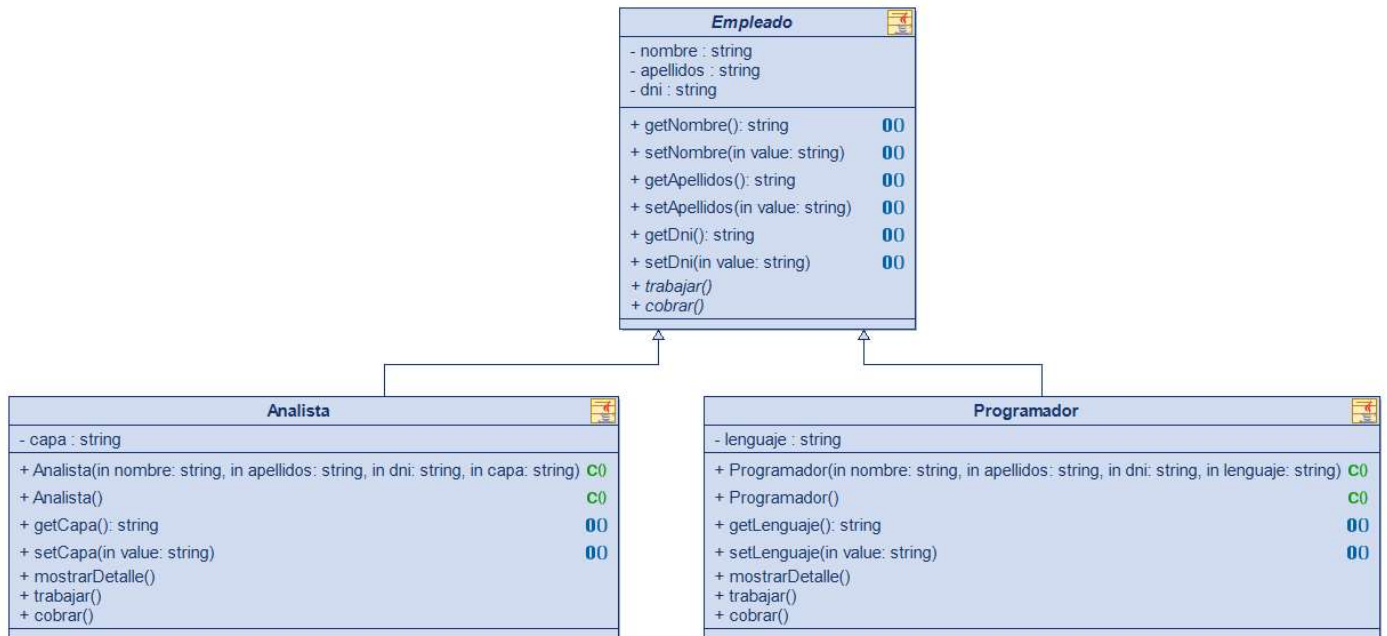


Implementar en PHP las clases con lo necesario para poder probar su ejecución y prueba.

10. Crear una relación de herencia a partir de una clase abstracta. Para lo cual, modelar un sistema donde se tienes estas clases.

- Empleado:
 - Con estos atributos:
 - nombre, cadena.
 - Apellidos, cadena.
 - dni, cadena
 - Con estos métodos:
 - Establecer nuevos valores con los métodos set.
 - Obtener valores con los métodos get.
 - trabajar.
 - cobrar.
- Analista: hereda de empleado implementando los métodos
 - Con estos atributos:
 - capa, cadena.
 - Con estos métodos:
 - Constructor de la clase que reciba los valores iniciales del mismo en su creación, sobrecargado sin valores.
 - Establecer nuevos valores con los métodos set.
 - Obtener valores con los métodos get.
 - Implementa los métodos del padre.
 - mostrarDetalle, que muestra toda la información del programador.
- Programador: hereda de empleado implementando los métodos
 - Con estos atributos:
 - lenguaje, cadena.
 - Con estos métodos:
 - Constructor de la clase que reciba los valores iniciales del mismo en su creación, sobrecargado sin valores.
 - Establecer nuevos valores con los métodos set.
 - Obtener valores con los métodos get.
 - Implementa los métodos del padre.
 - mostrarDetalle, que muestra toda la información del programador.

Con esta información, se ha diseñado y modelado las clases de la siguiente forma:



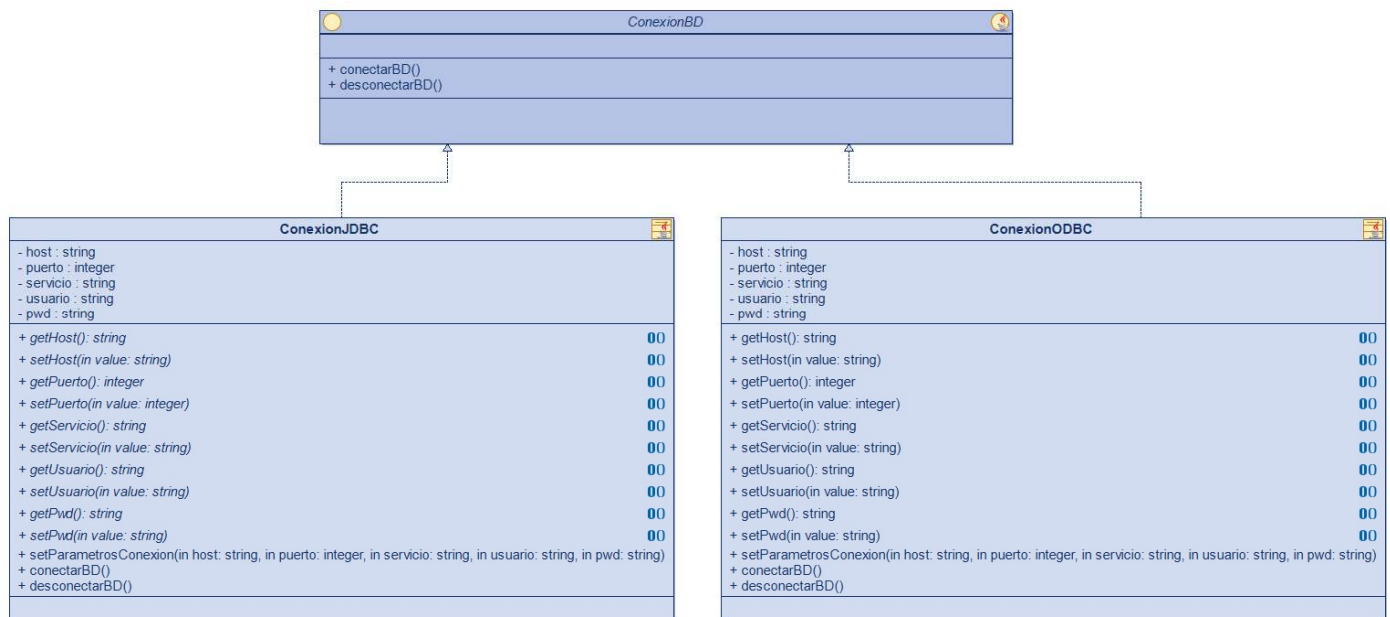
Implementar en PHP las clases con lo necesario para poder probar su ejecución y prueba.

11. Modelar la implementación de la interfaz siguiente:

- ConexionBD (interfaz):
 - Con estos métodos:
 - conectarBD.
 - desconectarBD.
- ConexionJDBC (implementación de la interfaz con JDBC):
 - Con estos atributos:
 - host, cadena.
 - puerto, entero.
 - servicio, cadena.
 - usuario, cadena.
 - pwd, cadena.
 - Con estos métodos:
 - Constructor de la clase que reciba los valores iniciales del mismo en su creación.
 - Establecer nuevos valores con los métodos set.
 - Obtener valores con los métodos get.
 - setParmetros que reinicie los valores de conexión.
 - conectarBD.
 - desconectarBD.
 - Implementa los métodos del padre.
 - mostrarDetalle, que muestra toda la información del programador.
- ConexiónODBC (implementación de la interfaz con ODBC):
 - Con estos atributos:
 - host, cadena.
 - puerto, entero.
 - servicio, cadena.
 - usuario, cadena.
 - pwd, cadena.
 - Con estos métodos:
 - Constructor de la clase que reciba los valores iniciales del mismo en su creación.
 - Establecer nuevos valores con los métodos set.
 - Obtener valores con los métodos get.
 - setParmetros que reinicie los valores de conexión.
 - conectarBD.
 - desconectarBD.

- Implementa los métodos del padre.
- mostrarDetalle, que muestra toda la información del programador.

Con esta información, se ha diseñado y modelado las clases de la siguiente forma:



Implementar en PHP las clases con lo necesario para poder probar su ejecución y prueba.

12. (Entregable) La empresa ModeloReferencia administra espacios de trabajo en varios edificios corporativos. Cada sitio corresponde a un espacio físico (puesto, sala, despacho...), identificado de forma única y con información sobre su ubicación y características básicas.

Existen dos tipos especializados de sitios:

- Sitios abiertos: puestos de trabajo individuales o compartidos.
- Salas: espacios para reuniones, formación o trabajo colaborativo.

Estos sitios están representados en una base de datos Oracle con las siguientes tablas, en las que sitios es la generalización de las tablas especializadas abiertos y salas:

- sitios (tabla base, con atributos comunes).

```

CREATE TABLE sitios (
    identificador NUMBER(6) NOT NULL,
    nombre VARCHAR2(50) NOT NULL,
    planta VARCHAR2(10) NOT NULL,
    descripcion VARCHAR2(250) NOT NULL,
    es_ventana CHAR(1) DEFAULT 'S' NOT NULL,
    tipo CHAR(1) DEFAULT 'A' NOT NULL,
    CONSTRAINT sit_pk PRIMARY KEY (identificador)
);
  
```

- abiertos (especialización para sitios abiertos).

```

CREATE TABLE abiertos(
    identificador NUMBER(6) NOT NULL,
    tipo_mesa VARCHAR2(1) DEFAULT 'R' NOT NULL,
    tiene_cajonera CHAR(1) DEFAULT 'S' NOT NULL,
    num_empleado NUMBER(6),
    CONSTRAINT abi_pk PRIMARY KEY (identificador),
    CONSTRAINT abi_fk1 FOREIGN KEY (identificador) REFERENCES sitios (identificador)
);
  
```

- salas (especialización para salas):

```
CREATE TABLE salas (
    identificador NUMBER(6) NOT NULL,
    cantidad_puestos NUMBER(3) NOT NULL,
    tiene_pizarra CHAR(1) DEFAULT 'S' NOT NULL,
    tiene_proyector CHAR(1) DEFAULT 'S' NOT NULL,
    CONSTRAINT sal_pk PRIMARY KEY (identificador),
    CONSTRAINT sal_fk1 FOREIGN KEY (identificador) REFERENCES sitios (identificador)
);
```

La empresa desea crear un modelo de clases en PHP orientado a objetos que refleje esta jerarquía, de forma que sea posible:

- Representar la información de los distintos tipos de sitios en memoria.
- Reutilizar código común mediante herencia.
- Implementar constructores, getters/setters y métodos para mostrar la información.

Para lo cual, se ha realizado el siguiente modelo de clases:

- Clase Sitio (base)
 - Atributos: identificador, nombre, planta, descripción, esVentana, tipo.
 - Métodos:
 - Constructor con parámetros obligatorios.
 - Getters y setters.
 - mostrarInfo(): imprime atributos comunes.
- Clase Abierto (hereda de Sitio):
 - Atributos: tipoMesa, tieneCajonera, numEmpleado.
 - Métodos:
 - Constructor: ha de llamar al constructor de la clase de la que hereda.
 - Getters y setters.
 - mostrarInfo(): imprime atributos heredados + propios.
- Clase Sala (hereda de Sitio):
 - Atributos: cantidadPuestos, tienePizarra, tieneProyector.
 - Métodos:
 - Constructor: ha de llamar al constructor de la clase de la que hereda.
 - Getters y setters.
 - mostrarInfo(): imprime atributos heredados + propios.

Implementar en PHP la clase con lo necesario para poder probar su ejecución y prueba. Utilizar tipado fuerte (tipos en las variables, parámetros y métodos).